



STUDENT NAME AMON AQUILA WILFRED
REGISTRATION NUMBER BSCCS/2022/31309
COURSE BACHELOR OF SCIENCE IN COMPUTER SCIENCE
UNIT CODE BIT3208
UNIT NAME ADVANCED WEB DESIGN AND DEVELOPMENT
LECTURER NAME MICHAEL NYORO

Project Title

NextBit computers:

A Trust Infrastructure and Automated Escrow Platform for Computer Retail

SELECTED TECHNOLOGIES

Frontend	Backend	Database
Next.js (React Framework) Tailwind CSS	Rust (Axum Framework) Python (Fast API)	PostgreSQL (Neon hosted) M-Pesa Daraja API

ONLINE PORTFOLIO

Vercel link: <https://nextbit-computers.vercel.app>

GITHUB

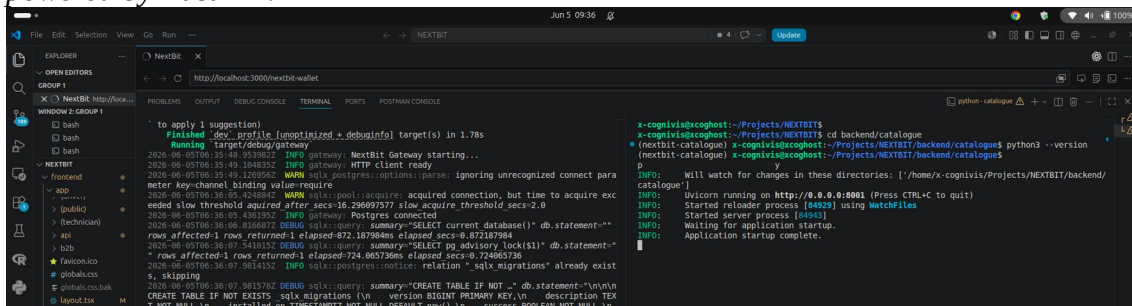
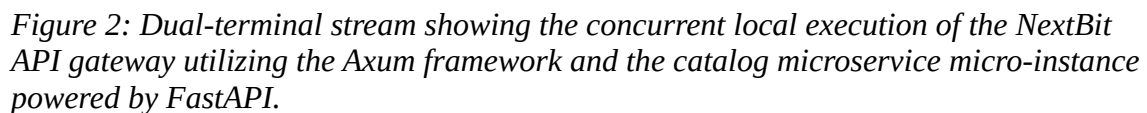
GitHub Repository: https://github.com/AquilaWilfred/nextbit_computers

Table of Contents

A Trust Infrastructure and Automated Escrow Platform for Computer Retail.....	1
Week 1:.....	3
Local Environment Setup.....	3
Installation and Configuration of Local Development Environment.....	3
My Reflection.....	5
Week 2:.....	6
Wireframes and GUI Design.....	6
User Interface Planning and System Design.....	6
Interactive High-Fidelity GUI Prototype.....	10
My Reflection.....	10
Week 3:.....	11
JavaScript and Backend Foundations.....	11
Client-Side Interactivity Logic and Backend API Routing Architecture.....	11
My Reflection:.....	14
Week 4:.....	15
Database Modeling and API Integration.....	15
Relational Database Architecture, Migrations, and Dynamic Component Data Binding.....	15
My Reflection.....	18
Week 5:.....	19
User Authentication and Security Architecture.....	19
Implementation of Secure Session Management, Token Authentication, and Route Protection	
.....	19
Login Endpoint and Authentication Token Generation.....	20
My Reflection.....	22
PORTFOLIO LINK.....	22
GITHUB LINK.....	22

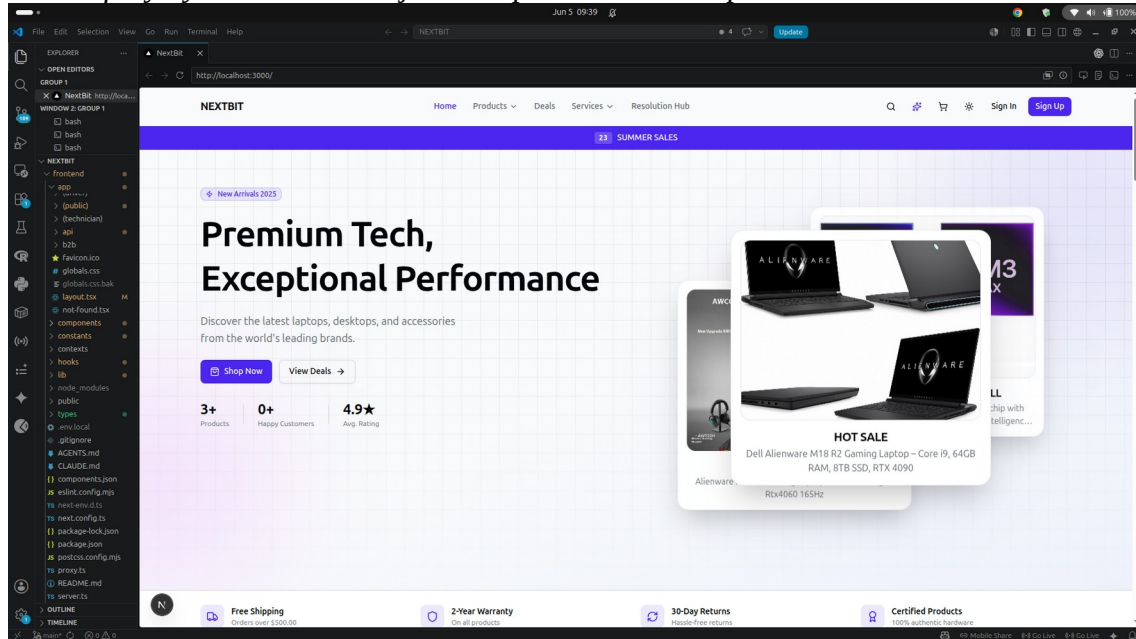
Installation and Configuration of Local Development Environment

Figure 1: Verification of the local system toolchains on a 64-bit Linux environment, showing successful installation of the Rust compiler (rustc/cargo) and Python 3 runtime required for NextBit services.



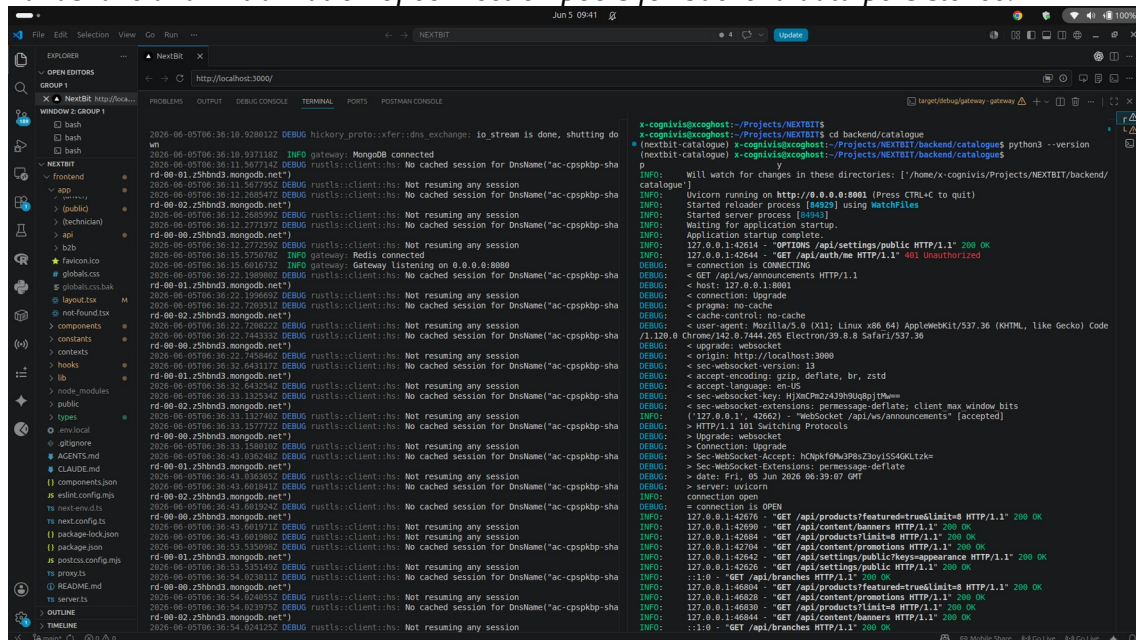
Next.js Frontend Localhost Initialization

Figure 3: Web browser view rendering the Next.js client-side application compiled successfully by the local Node.js development server on port 3000.



Database Connection Initialization Log

Figure 4: Local storage database state log stream confirming successful network handshake and initialization of connection pools for backend data persistence.



My Reflection

This week, I successfully initialized a multi-service local environment on a 64-bit Linux platform to support the NextBit ecosystem. The architecture separates the frontend client (Next.js) from the backend architecture, which utilizes a high-performance Rust (Axum) API gateway alongside Python (FastAPI) for modular business logic. I encountered minor port conflicts when running multiple concurrent servers, which I resolved by explicitly binding services to separate local ports. The verification screenshots confirm that both backend runtimes compile correctly, communicate via HTTP protocols, and successfully establish a stable connection pool to the local database layer.

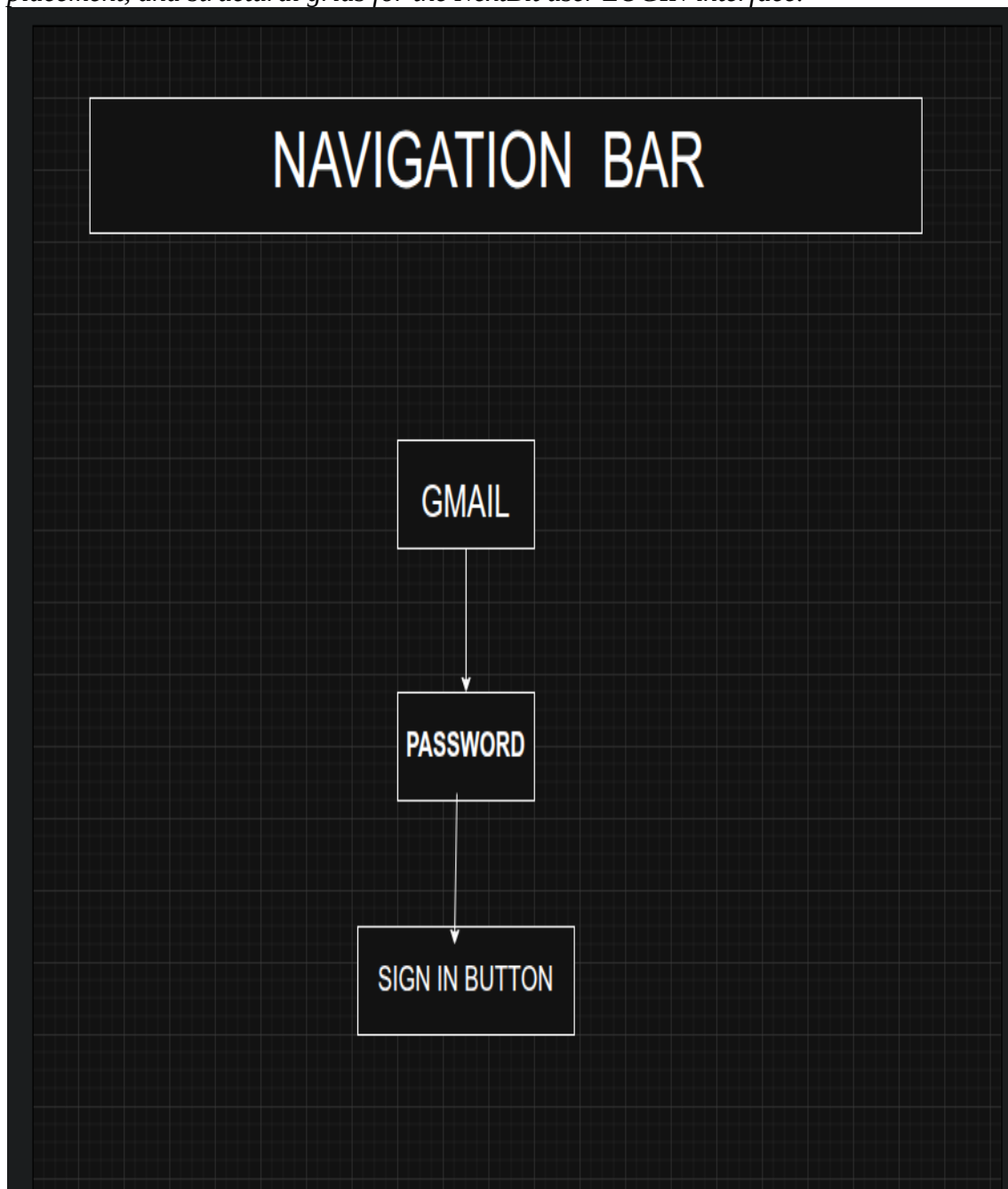
Week 2

Wireframes and GUI Design

User Interface Planning and System Design

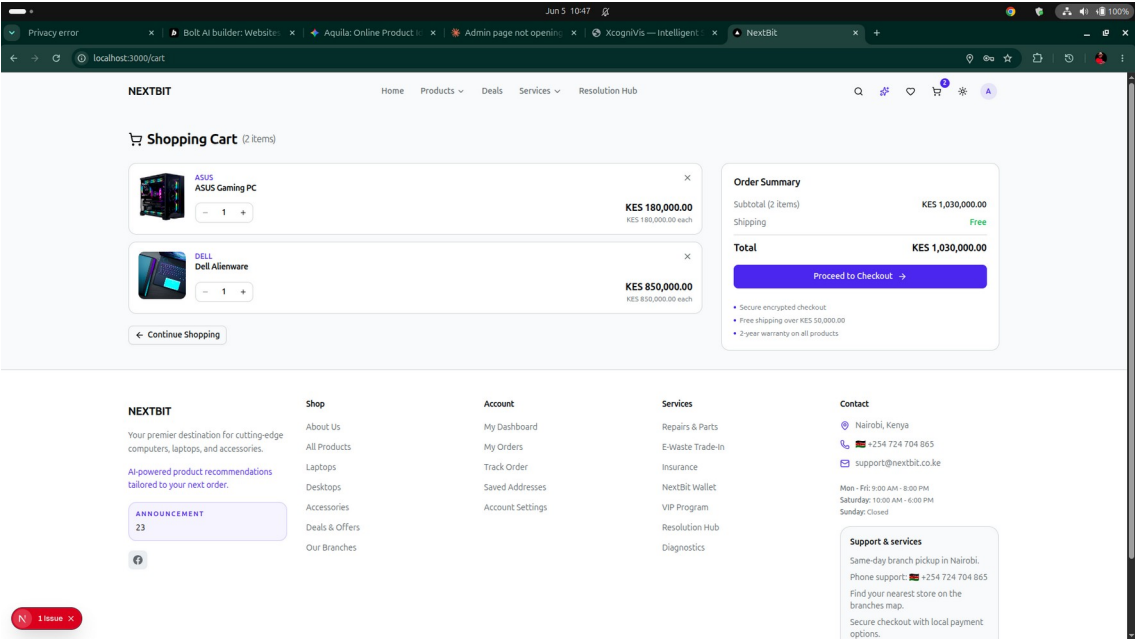
Hand-Drawn Conceptual Wireframe (Landing & Authentication)

Figure 5: Low-fidelity drawn layout blueprint mapping the visual hierarchy, content placement, and structural grids for the NextBit user LOGIN interface.



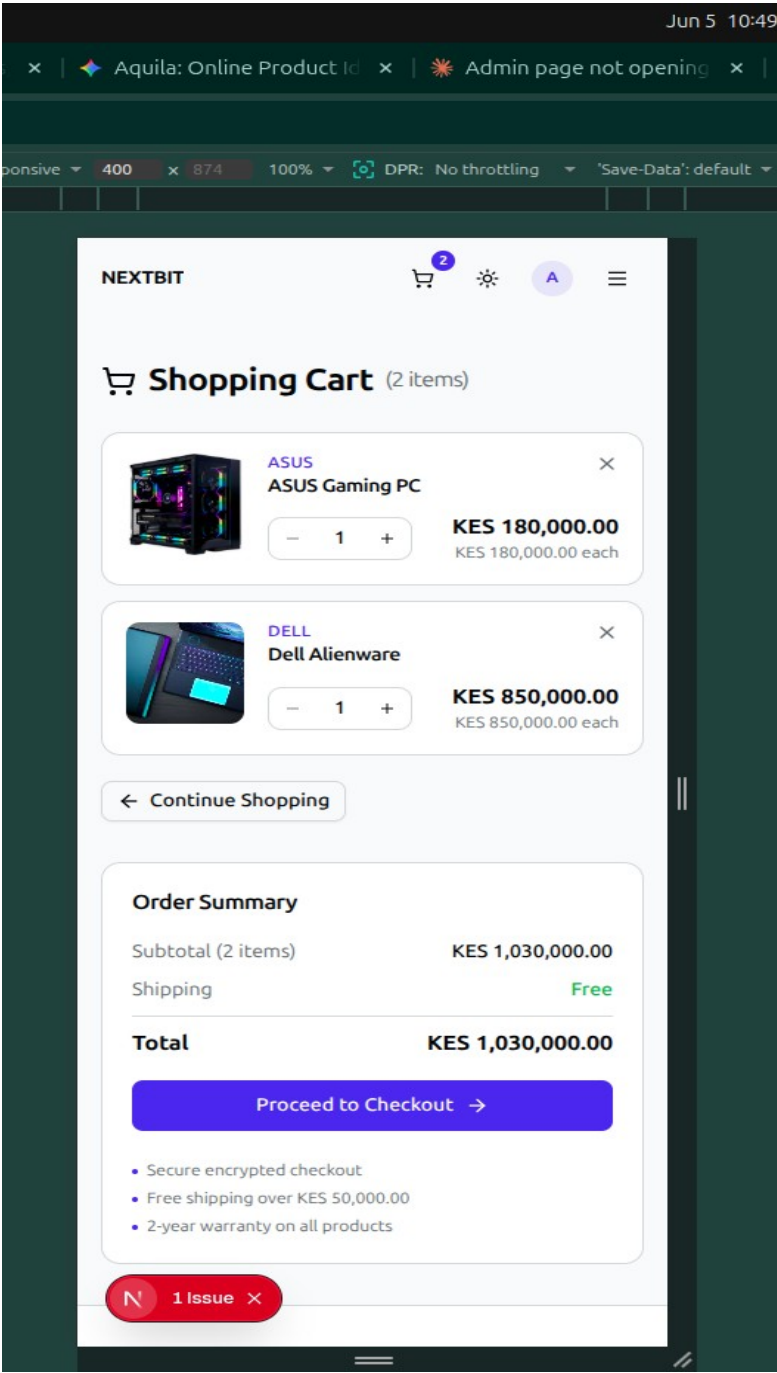
Buyer Dashboard Navigation and Layout Design

Figure 6: Conceptual wireframe design for the authenticated user control panel, emphasizing dynamic sidebar scaling and accessibility for tracking local hardware assets.



Responsive Mobile View Mockup

Figure 7: UI breakpoint design illustrating cross-platform scaling mechanics to ensure clean user experience flows across mobile screen dimensions.

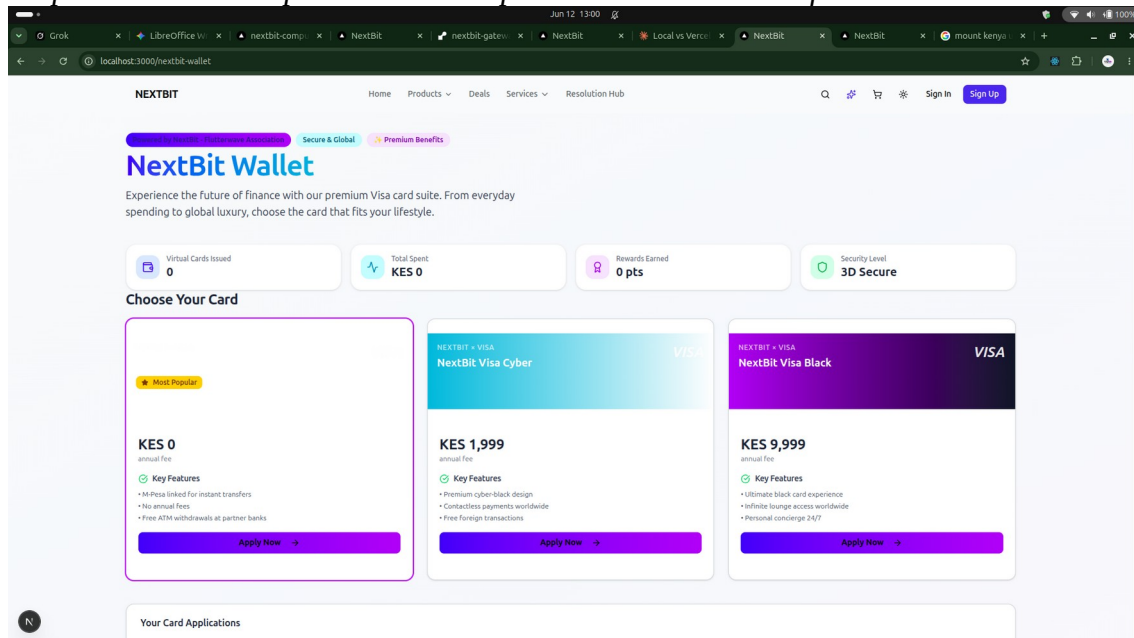


Typography, Color Palette, and Design Tokens

These three colors are the visual identity of NextBit. . Each color has one primary psychological job.

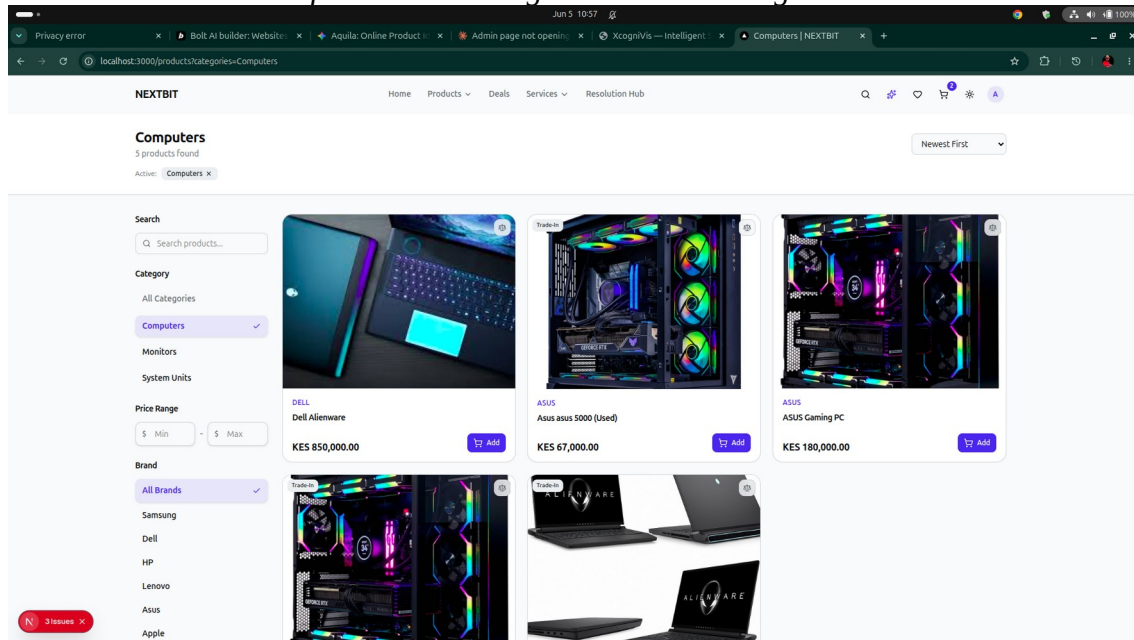
#00F2FF	#0055FF	#00D9C0
Primary Cyan	Deep Cobalt	Electric Teal
Action · Buttons · QR badge · Scan success	Trust · Nav bars · Logbook header · Certifications	Growth · M-Pesa balance · Bond status · Sanitized

Figure 8: UI styling design matrix defining the semantic typography sizing scales and deep-tech dark mode palette tokens implemented to drive user focus.



Interactive High-Fidelity GUI Prototype

Figure 9: High-fidelity interactive layout prototype modeling the automated escrow transaction status view prior to rendering active backend logic blocks.



My Reflection

This week, I established the user experience patterns and semantic design systems for NextBit. The interface was intentionally strategized to optimize structural layout visibility, allowing users to safely track high-value escrow states with minimal cognitive clutter. By utilizing modular grid lines and responsive side-navigation layouts, the application ensures high scalability as hardware validation reporting fields expand. Design tokens were mapped tightly to Tailwind CSS formatting classes, bridging the gap between design theory and implementation to accelerate future client-side routing and programmatic rendering phases.

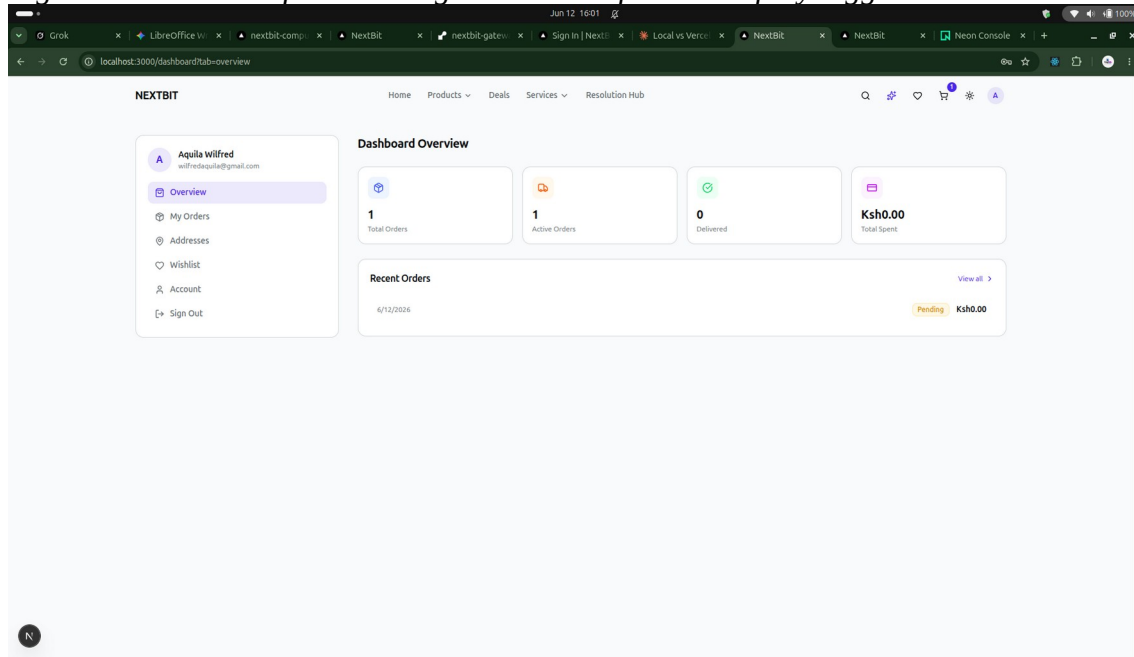
Week 3

JavaScript and Backend Foundations

Client-Side Interactivity Logic and Backend API Routing Architecture

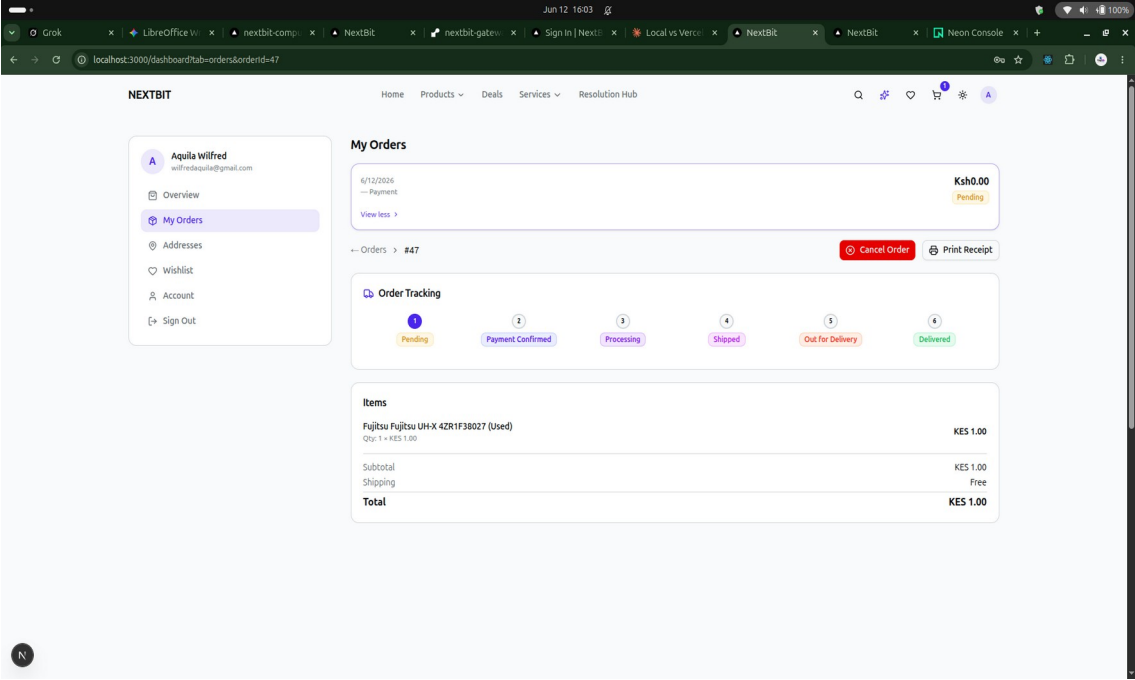
Client-Side UI Dashboard

Figure 10: User interface showing dashboard of a successfully logged in client



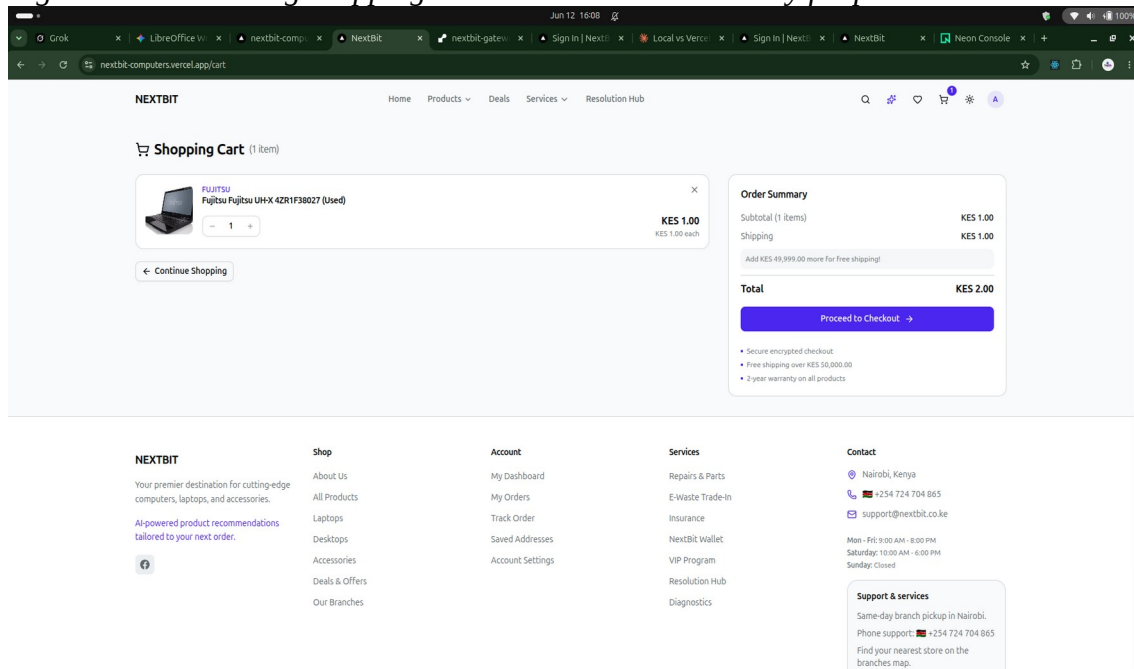
Client-Side navigation and redirection using bars

Figure 11: UI shows an interactive view of my orders in the image



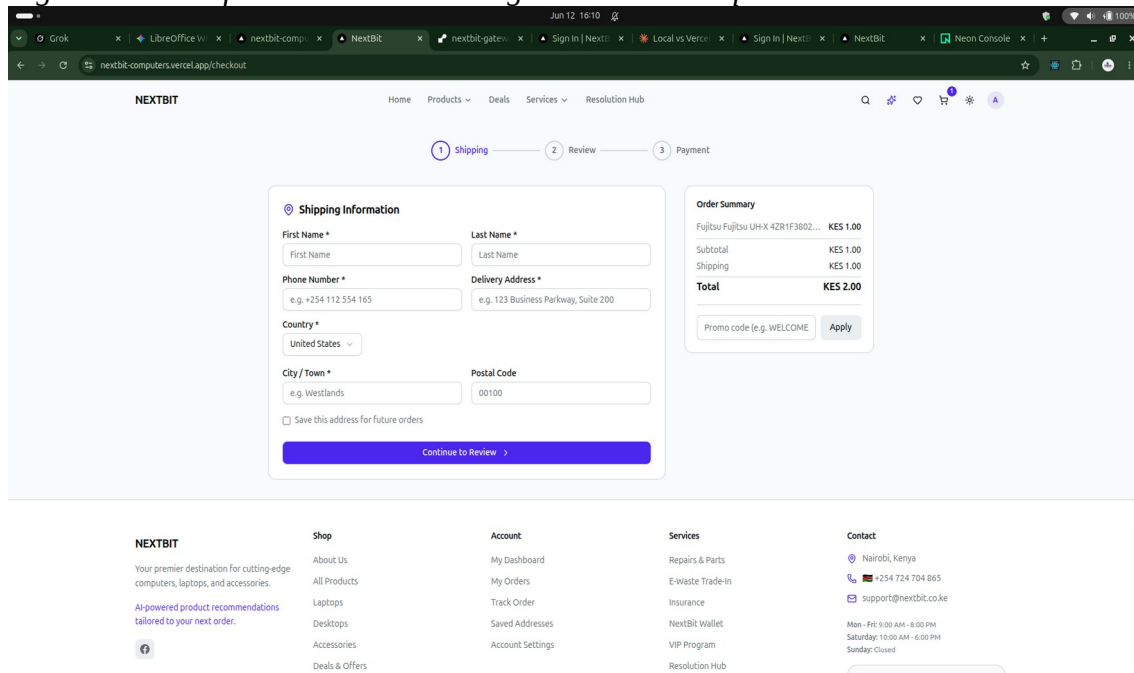
UI navigation to the Shopping cart

Figure 12: UI showing shopping cart with 1 item listed ready for purchase



Fronted UI for Shipping Information

Figure 13: Core fields are marked using * to indicate required



Payment Escrow initiation

Figure 14: M-Pesa STK Push ready for Order **ORD-3FBCE41C** — Total: **KES 2.00**

Store

Home Products Deals Services Resolution Hub

Shipping Review **Payment**

Complete Payment
Order **ORD-3FBCE41C** — Total: **KES 2.00**

M-Pesa STK Push
Enter your M-Pesa phone number. You'll receive a prompt to enter your PIN.

M-Pesa Phone Number
+254 112 554 165

Send STK Push

Order Summary
Fujitsu Fujitsu UH-X 42R1F3802... **KES 1.00**
Subtotal **KES 1.00**
Shipping **KES 1.00**
Total KES 2.00

Promo code (e.g. WELCOME) **Apply**

Store
Your premier destination for cutting-edge computers, laptops, and accessories.
AI-powered product recommendations and special offers tailored to your next...

Shop
About Us
All Products
Laptops
Desktops
Accessories
Deals & Offers
Our Branches

Account
My Dashboard
My Orders
Track Order
Saved Addresses
Account Settings

Services
Repairs & Parts
E-Waste Trade-In
Insurance
NextBit Wallet
VIP Program
Resolution Hub
Diagnostics

Contact
123 Innovation Drive, Nairobi, Kenya
+254 724 704 865
support@company.com
Mon - Fri: 9:00 AM - 8:00 PM
Saturday: 10:00 AM - 6:00 PM

Support & services
Same-day branch pickup in Nairobi.
Phone support: +254 724 704 865
Find your nearest store on the branches map.
Secure checkout with local payment

My Reflection:

This week focused on connecting front-end processing logic with stable back-end structures. I implemented robust client-side validation using structured JavaScript patterns to parse input metrics safely, reducing redundant server load. On the server layer, I structured scalable endpoint routes designed to process incoming network payloads asynchronously. The core challenge involved mapping flexible client requests to strict relational database schemas, which I resolved by utilizing highly explicit data transfer schemas.

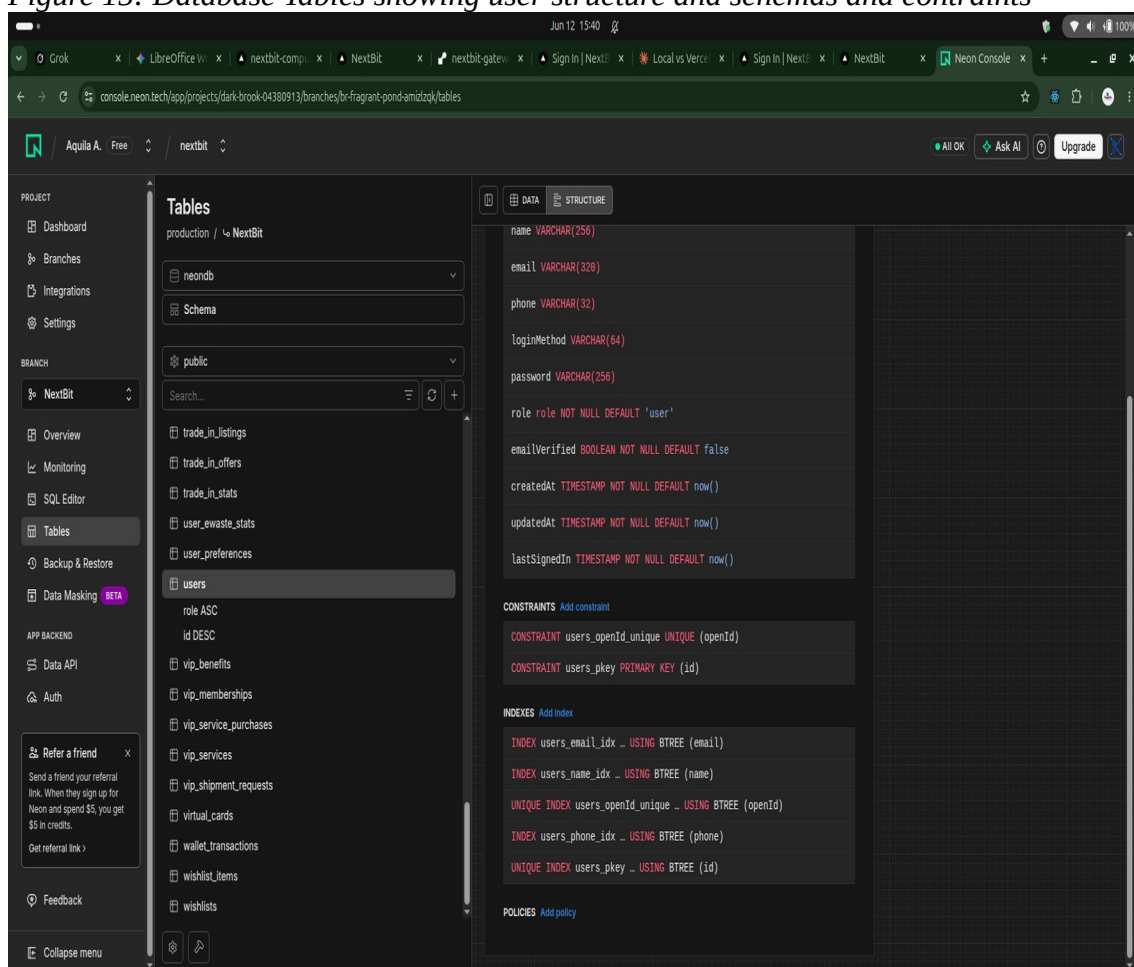
Week 4

Database Modeling and API Integration

Relational Database Architecture, Migrations, and Dynamic Component Data Binding

Database Table Schema and Constraints Design

Figure 15: Database Tables showing user structure and schemas and constraints



Database Migration Execution or SQL Seed Script

Figure 16: Successfully installed sql migrations for the escrow payments

version bigint	description text	installed_on timestamp ...	success boolean	checksum bytea	execution_time bigint
20260419193959	init	2026-04-20 19:07:51.48...	TRUE	\x1b5b31ed670ada6928df...	1987209830
20260419193960	devices	2026-05-08 17:12:57.68...	TRUE	\xe1ad1327906f1e3b503d...	1481884510
20260500000000	create stub users orde...	2026-05-24 09:00:11.63...	TRUE	\xe9b9d059d1db4e623a4d...	1432634821
20260501000000	escrow	2026-05-24 09:07:48.55...	TRUE	\x13157220e1222217b00b...	1429322086
20260502000001	wallet	2026-05-24 09:19:16.27...	TRUE	\xfea9a6538b5fe5b7a8b9...	1444374668
20260503000000	daraja escrow	2026-05-29 12:26:57.83...	TRUE	\x051b6d3dbc56227c2968...	1640027814

Frontend Data Fetching and Feeding Service Component

Figure 17: Client-side engine component implementing asynchronous network services to call data from local API gateway routes that involve listing a devise into the market.

Jun 12 19:52

Crok

LibreOffice W

nextbit-comp

NextBit

nextbit-gatew

Sign in | NextB

Local vs Verce

Sign in | NextB

NextBit

Neon Console

nextbit-computers.vercel.app/listings

HomeProductsDealsServicesResolution Hub

NextBIT

Search

Heart

Shopping Cart

Settings

Profile

List Your Device

Fill in the details below to start selling on NextBit Marketplace

Device Type *

Laptop

Brand *

Select brand

Model *

e.g. iPhone 14 Pro, Galaxy S23

Asking Price (KES) *

e.g. 45,000

Condition *

Excellent

Good

Fair

Like new, no visible wear

Minor wear, fully functional

Visible wear, fully functional

Device Photos (Max 5)

Click to upload or drag and drop

PNG, JPG up to 5MB each. 0/5 selected.

Location

Nairobi

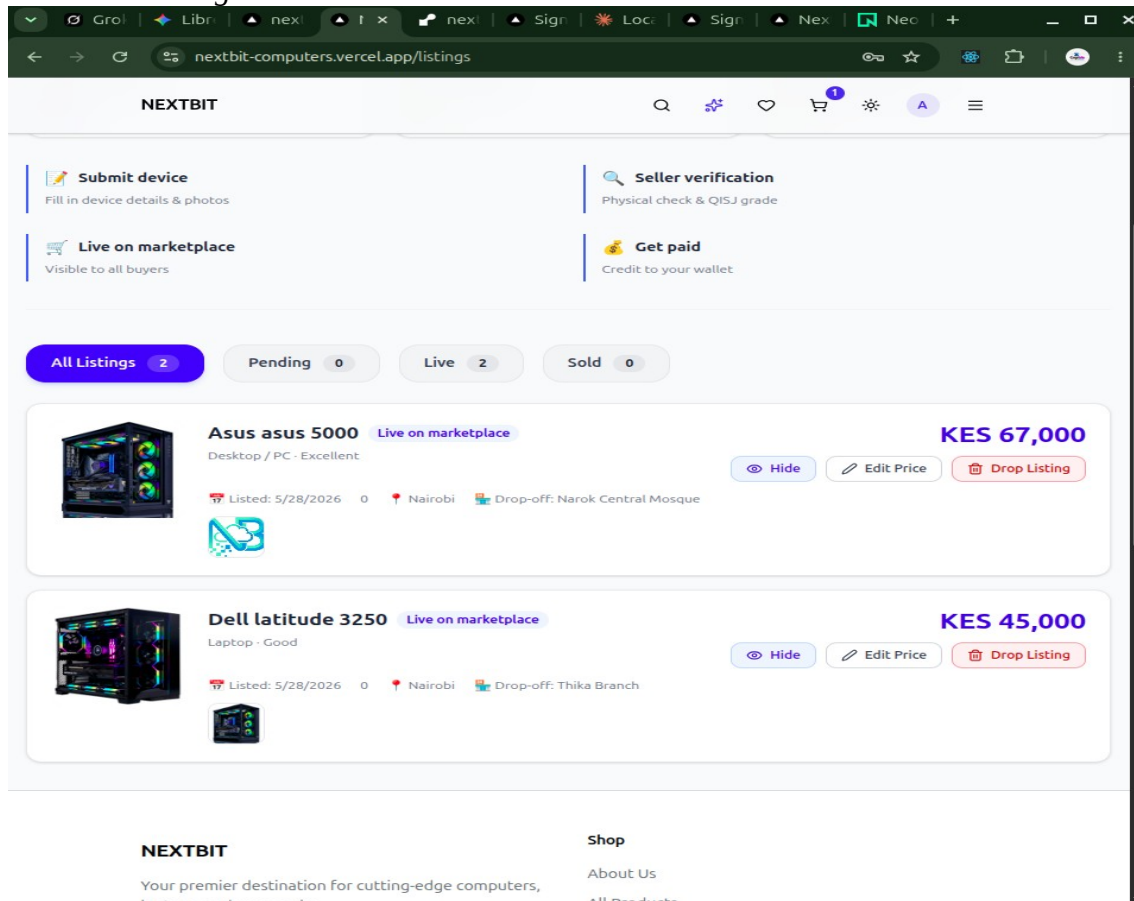
Detect

Drop-off Branch *

Select branch

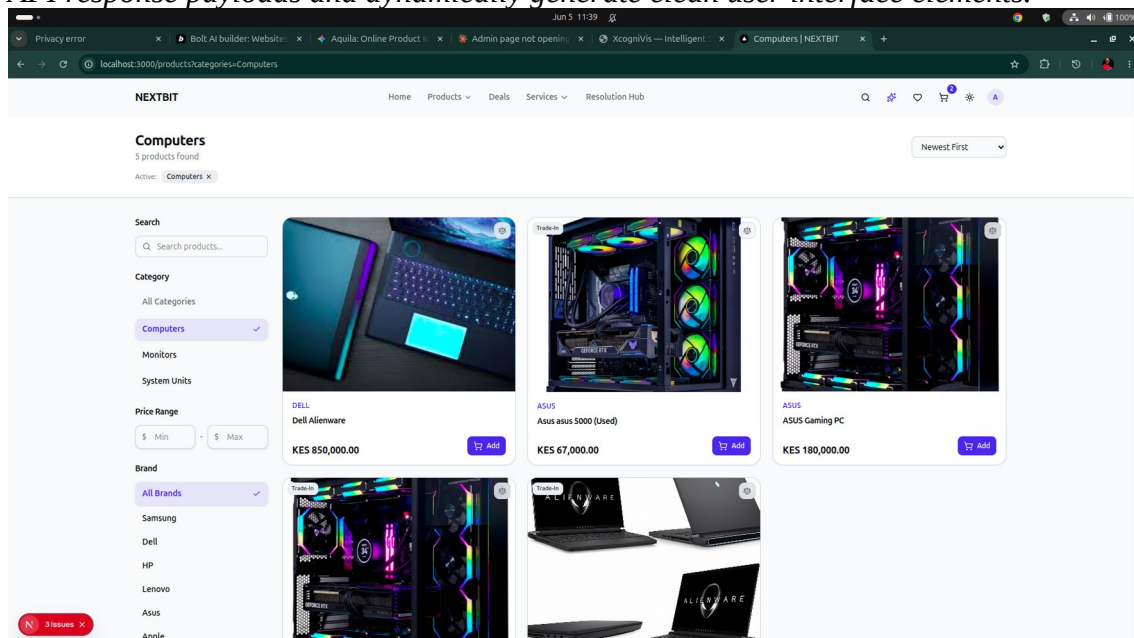
Backend Controller Mapping Query Outputs to JSON

Figure 18: Server-side route controller executing internal queries to extract, format, and serialize data records into clean JSON response structures which are then rendered as shown in image



Dynamic Client-Side Interface Component Rendering

Figure 19: Frontend view component utilizing array mapping scripts to loop through API response payloads and dynamically generate clean user interface elements.



My Reflection

This week focused on stabilizing the application's data layer and syncing it dynamically with client-side interface elements. I designed structured relational tables to house application entities, enforcing data boundaries at the storage level. On the client side, I implemented asynchronous service workers to poll state updates from the backend microservices. Managing network states during asynchronous loading presented minor challenges, which I mitigated by incorporating precise interface models. The evidence demonstrates an operational flow where backend queries are formatted into typed payloads and translated cleanly into dynamic interface elements.

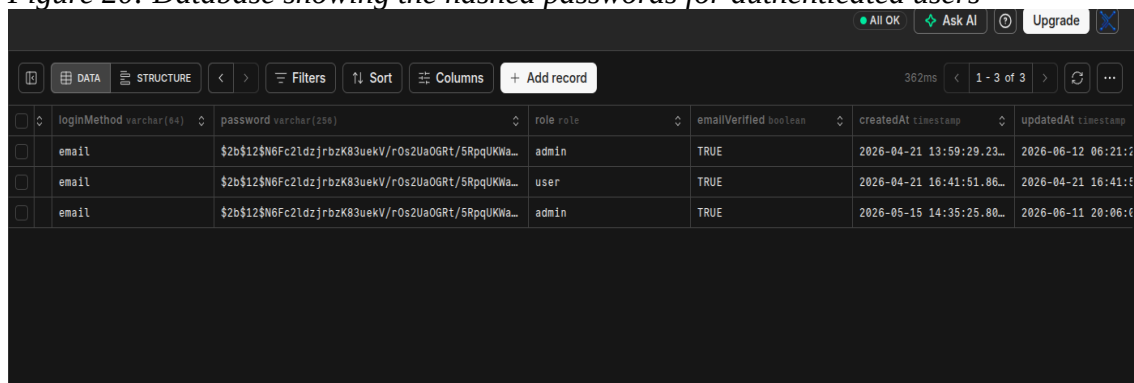
Week 5

User Authentication and Security Architecture

Implementation of Secure Session Management, Token Authentication, and Route Protection

User Password Hashing and Cryptographic Processing

Figure 20: Database showing the hashed passwords for authenticated users

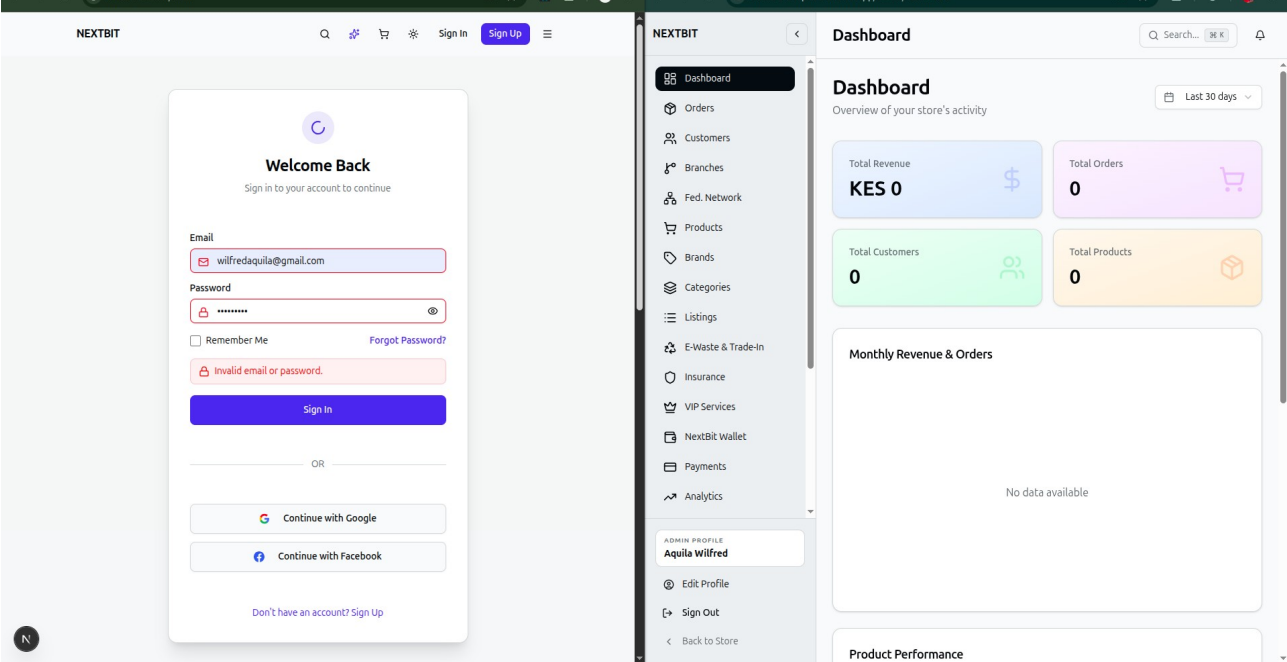


The screenshot shows a database table with the following columns: loginMethod, password, role, emailVerified, createdAt, and updatedAt. The table contains three records. The first record is an admin user with a hashed password. The second record is a user with a hashed password. The third record is an admin user with a hashed password. The passwords are all the same, indicating a test environment.

	loginMethod varchar(64)	password varchar(256)	role role	emailVerified boolean	createdAt timestamp	updatedAt timestamp
☐	email	\$2b\$12\$N6Fc2ldzjrbzK83uekV/r0s2Ua0GRt/5RpqUKWa...	admin	TRUE	2026-04-21 13:59:29.23...	2026-06-12 06:21:7...
☐	email	\$2b\$12\$N6Fc2ldzjrbzK83uekV/r0s2Ua0GRt/5RpqUKWa...	user	TRUE	2026-04-21 16:41:51.86...	2026-04-21 16:41:5...
☐	email	\$2b\$12\$N6Fc2ldzjrbzK83uekV/r0s2Ua0GRt/5RpqUKWa...	admin	TRUE	2026-05-15 14:35:25.80...	2026-06-11 20:06:6...

Login Endpoint and Authentication Token Generation

Figure 21: login authentication failure and success login images using similar images but different passwords. First image failed due to wrong inputs



Protected Route Middleware / Guard Logic

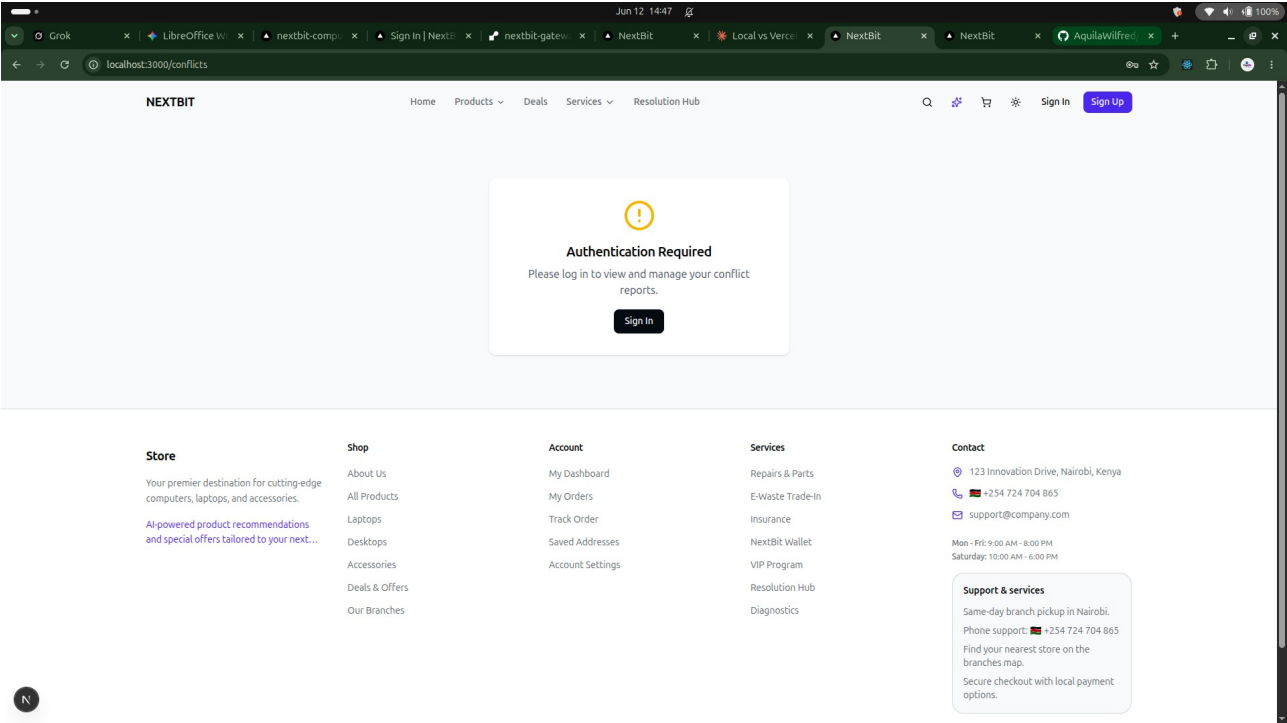


Figure 22: Route security interceptor output evaluating incoming request headers to block unauthenticated access to sensitive system resources.

Frontend Auth State Provider / Context Client

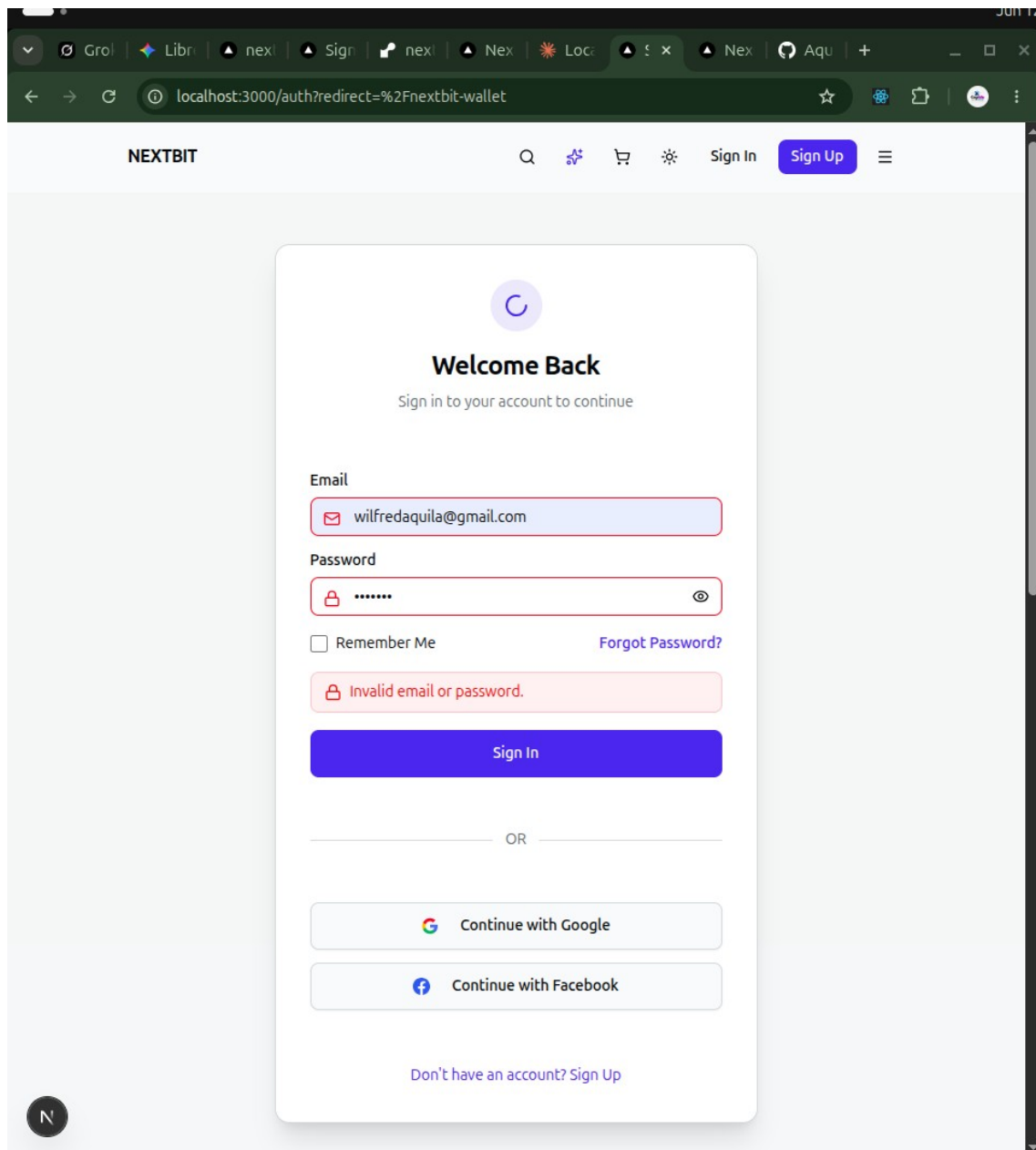
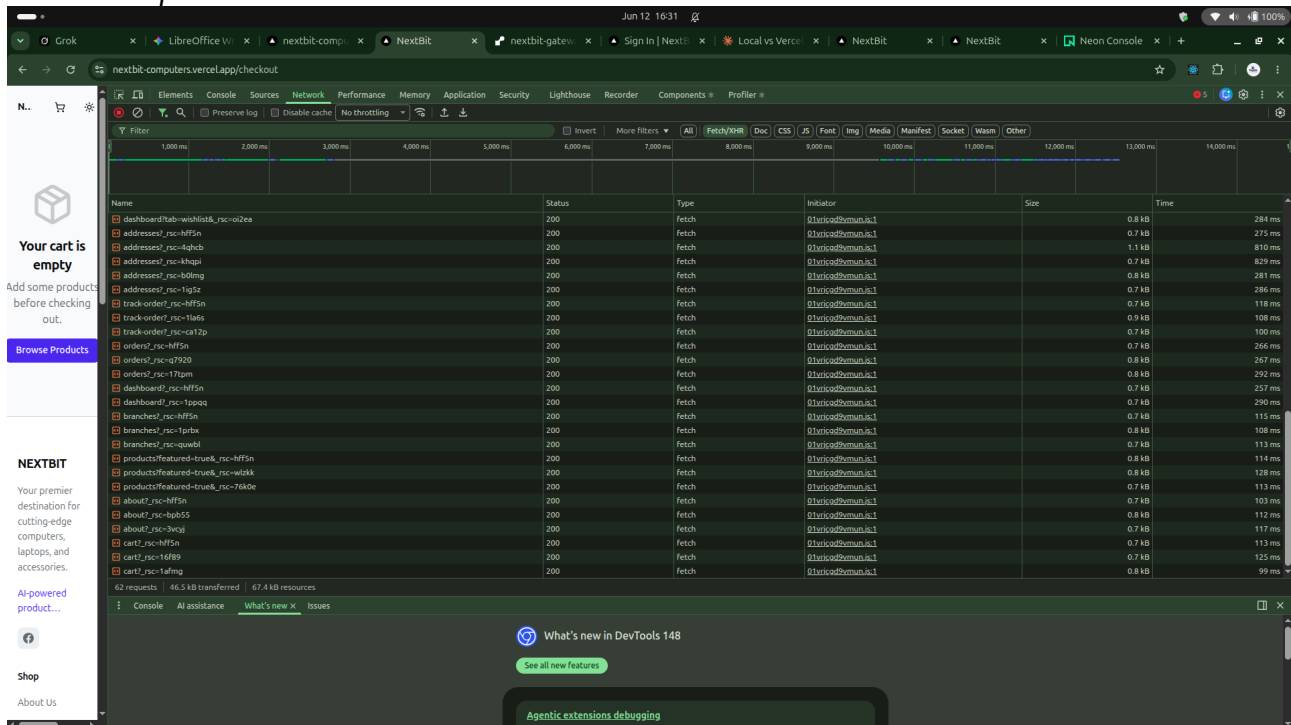


Figure 23: Client-side context provider state managing persistent user authentication tokens and application access blocked on wrong input

Network Request Interceptor / Secure Fetch Call

Figure 24: Network data service inspection browser tool showing successful and failing requests. The successful ones show 200 OK



My Reflection

This week focused on implementing strict application security models and stateless user identity management. I engineered backend middleware components to parse and validate incoming cryptographic keys, ensuring that secure database records remain completely walled from unauthenticated access layers. Handling authentication state initialization on client reloads presented minor challenges, which I addressed by linking token validation logic directly into the frontend startup cycle. The resulting mechanics establish a resilient security boundary where user sessions are verified dynamically at the gateway before any transactional microservice processing can take place.

PORTFOLIO LINK

Vercel link: <https://nextbit-computers.vercel.app>

GITHUB LINK

GitHub Repository: https://github.com/AquilaWilfred/nextbit_computers